МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра Програмної інженерії

Звіт
з лабораторної роботи №5
з дисципліни: «Безпека програм та даних»
з теми: «Реалізація платіжної форми веб-сайту та захист електронних транзакцій»

Виконали:
ст. гр. ПЗПІ-23-3
Чуваєв А. О. (вар. 16)
ст. гр. ПЗПІ-23-3
Світенко С. М. (вар. 13)

Перевірив:
Старший викладач кафедри ПІ
Олійник О. О.

Харків – 2026

5 РЕАЛІЗАЦІЯ ПЛАТІЖНОЇ ФОРМИ ВЕБ-САЙТУ ТА ЗАХИСТ ЕЛЕКТРОННИХ ТРАНЗАКЦІЙ

5.1 Мета роботи

Дослідити криптографічні механізми перевірки автентичності та цілісності транзакцій електронної комерції під час оплати платіжними картками, інтегрувати платіжний шлюз LiqPay у тестовому режимі (Sandbox), реалізувати серверний механізм обробки статусів оплат на базі ASP.NET Core Minimal API із контролем залишків товарів на складі та захистом від повторних списань (ідемпотентність), а також забезпечити безпечну автентифікацію клієнтів за стандартом OAuth 2.0 (RFC 6749).

5.2 Індивідуальне завдання та технічні вимоги

Робота виконується бригадою у складі двох студентів групи ПЗПІ-23-3:

- **Студент 1:** Чуваєв Артем Олександрович (Варіант 16).
- **Студент 2:** Світенко Софія Миколаївна (Варіант 13).

Технічні вимоги до програмної реалізації:

а) **Сервіс електронної комерції та контроль складу:** розробити інтернет-магазин із реальними товарами (автомобільний салон) та обліком залишків. Для цільового автомобіля **ЗАЗ-968М** встановити початковий залишок **3 шт.** Після успішної оплати залишок має зменшуватися до **2 шт.**, а після неуспішної або відхиленої транзакції – залишатися рівним **3 шт.**

б) **Інтеграція платіжної системи:** використати готовий платіжний шлюз (LiqPay) у тестовому режимі (**Sandbox**), що уможливлює проведення безпечних транзакцій без витрати реальних фінансових коштів.

в) **Криптографічний підпис форми та транзакцій:** забезпечити розрахунок та перевірку цифрового підпису згідно зі специфікацією $\mathrm{signature} = \mathrm{base64}\big(\mathrm{sha1}\big(S_{\mathrm{req}}\big)\big)$.

г) **Серверний обробник (Server-to-Server Callback / Webhook):** реалізувати ендпоінт для асинхронного прийому повідомлень про статус покупки з обов’язковою верифікацією підпису та блокуванням атак підробки параметрів (Tampering).

д) **Захист від повторних оплат:** реалізувати принцип ідемпотентності за унікальним «order_id».

е) **Автентифікація через OAuth 2.0 (вимога на відмінно):** реалізувати протокол авторизації RFC 6749 (Authorization Code Flow) із видачею токенів Bearer Access Token для облікових записів покупців.

5.3 Теоретичні відомості

5.3.1 Архітектура електронного еквайрингу та учасники платежу

Процес оплати платіжною карткою в мережі Інтернет базується на взаємодії кількох суб'єктів платіжної інфраструктури:

а) **Покупець (Cardholder):** власник банківської платіжної картки, який ініціює операцію купівлі товару.

б) **Торговець (Merchant):** веб-сайт або інтернет-магазин, що реалізує товари чи послуги.

в) **Платіжний шлюз (Payment Gateway):** спеціалізований захищений апаратно-програмний комплекс (наприклад, LiqPay, Stripe), що забезпечує збір платіжних реквізитів відповідно до стандарту PCI DSS, шифрування та передачу даних до платіжної мережі.

г) **Банк-еквайєр (Acquiring Bank):** фінансова установа, яка відкриває торговцю спеціальний мерчант-рахунок, забезпечує авторизацію платежів та здійснює перерахування коштів від покупців.

д) **Банк-емітент (Issuing Bank):** банк, що випустив платіжну картку покупця, відкрив відповідний картковий рахунок, перевіряє наявність коштів, проводить автентифікацію клієнта (3-D Secure) та списує кошти.

е) **Міжнародні платіжні системи (Payment Networks):** Visa, Mastercard, що виконують міжбанківський кліринг і маршрутизацію транзакцій.

5.3.2 Механізм криптографічного цифрового підпису LiqPay

У класичній архітектурі платіжна форма передається через браузер клієнта за протоколом HTTP POST. Оскільки клієнтська сторона не є довіреною, зловмисник

може перехопити та підмінити відкриті поля (наприклад, зменшити вартість товару з 15 000 грн до 1 грн або підмінити номер рахунку).

Для запобігання підробці використовується симетричний цифровий підпис запиту на основі секретного ключа торговця («private_key»), відомого лише серверу магазину та платіжному шлюзу. Підпис формується шляхом згортки конкатенованого рядка параметрів хеш-функцією SHA-1 з подальшим кодуванням у Base64:

$$\text{signature} = \text{base64}\big(\text{sha1}\big(S_{\text{req}}\big)\big), \tag{5.1}$$

де S_{req} – конкатенований рядок параметрів запиту:

$$\begin{aligned} S_{\text{req}} = {} & \text{private_key} \parallel \text{amount} \parallel \text{currency} \parallel \\ & \text{public_key} \parallel \text{order_id} \parallel \text{type} \parallel \text{description} \parallel \\ & \text{result_url} \parallel \text{server_url}. \end{aligned} \tag{5.2}$$

Платіжний шлюз повторно розраховує підпис за тією ж формулою, використовуючи зареєстрований «private_key» магазину. Якщо отриманий підпис не збігається з розрахованим хоча б на 1 біт, транзакція відхиляється.

5.3.3 Асинхронний серверний Callback (Webhook) та захист від підробки

Після завершення обробки транзакції банком-емітентом платіжний шлюз надсилає асинхронний POST-запит безпосередньо на «server_url» інтернет-магазину. Цей запит містить фінальний статус («status = success | sandbox | failure | wait_secure»), ID транзакції та цифровий підпис:

$$\text{signature} = \text{base64}(\text{sha1}(S_{\text{res}})), \tag{5.3}$$

де S_{res} – конкатенований рядок параметрів підтвердження:

$$\begin{aligned} S_{\text{res}} = {} & \text{private_key} \parallel \text{amount} \parallel \text{currency} \parallel \\ & \text{public_key} \parallel \text{order_id} \parallel \text{type} \parallel \text{description} \parallel \\ & \text{status} \parallel \text{transaction_id} \parallel \text{sender_phone}. \end{aligned} \tag{5.4}$$

Сервер магазину верифікує цей підпис перед внесенням будь-яких змін до бази даних замовлень або відвантаженням товару.

5.3.4 Авторизація за стандартом OAuth 2.0 (RFC 6749)

Стандарт OAuth 2.0 забезпечує делеговану автентифікацію без передачі пароля третім сторонам. Схема Authorization Code Grant включає:

а) Запит коду: «GET /oauth/ authorize?client_id=...&response_type=code&state=...»
б) Автентифікацію користувача та видачу короткоживучого одноразового коду «code».
в) Прямий обмін коду на маркер доступу: «POST /oauth/token» (з перевіркою «client_secret»).
г) Надання доступу до ресурсів за допомогою заголовка «Authorization: Bearer <token>».

5.4 Реалізація та архітектура системи

Система побудована на базі сучасного фреймворку **ASP.NET Core Minimal API (.NET 10)** та працює на мережевому порті **5090**:

Компонент	**Файл / Модуль**	**Функціональне призначення**
LiqPayService	LiqPayService.cs	Розрахунок SHA-1/Base64 підписів, інтеграція з https://www.liqpay.ua/api/3/checkout та Server-to-Server Status API
InventoryService	InventoryService.cs	Облік авто (ЗАЗ-968М, ЛуАЗ, Таврія), машина станів OrderState, ідемпотентність списання
OAuthService	OAuthService.cs	Провайдер RFC 6749 (Authorization Code, токени Bearer, профілі студентів Чуваєва та Світенко)
Web Server API	Program.cs	Minimal API ендпоінти каталогу, checkout, sync-status, callback webhook, OAuth та демо-режим
Конфігурація	appsettings.json	Безпечне зберігання ключів мерчанта LiqPay, режим Sandbox та публічний WebhookUrl
Web UI	index.html, app.js, style.css	Вітрина автосалону, кошик покупок, офіційне вікно оплати LiqPay, журнал замовлень

5.4.1 Складський облік та машина станів замовлення

Замовлення проходить наступні фази життєвого циклу:

– **Pending:** замовлення зареєстровано, сформовано платіжну форму з підписом, очікується перехід до шлюзу. Залишок товару: 3 шт.

- **Paid:** успішна оплата підтверджена валідним Webhook або Status API від шлюзу («status = success» або «status = sandbox»). Залишок товару зменшується: 3 → 2 шт.
- **Failed:** відхилення транзакції банком-емітентом («status = failure»). Стан фіксується, залишок не змінюється: 3 шт.
- **WaitSecure:** транзакція очікує перевірки 3-D Secure. Склад утримується без списання.

5.5 Практичні результати та протоколи тестування

Тестування комплексу проведено за допомогою вбудованого автоматизованого протоколу перевірки «--demo», а також через інтерактивний веб-інтерфейс.

5.5.1 Етап 1. Початковий стан складу автомобілів

Початкові складські залишки визначено відповідно до індивідуальних вимог:

- **ЗАЗ-968М «Запорожець»:** початковий залишок **3 шт.** (ціна 15.00 UAH).
- **ЛуАЗ-969М «Волинь»:** початковий залишок **2 шт.** (ціна 25.00 UAH).
- **ЗАЗ-1102 «Таврія Нова»:** початковий залишок **4 шт.** (ціна 35.00 UAH).

5.5.2 Етап 2. Автентифікація покупця за стандартом OAuth 2.0 (RFC 6749)

а) **Запит авторизації:** сформовано «client_id = autostore-web-app», «state = 037c1b22».

б) **Отримано Authorization Code:** «auth_code_7933940922d444daa1f8243896835181».

в) **Обмін токена на Token Endpoint:**
   - Маркер доступу: «Bearer bearer_token_79f3697...» (термін дії 3600 с).
   - Автентифікований користувач: **Чуваєв Артем Олександрович**, група ПЗПІ-23-3, варіант 16.

5.5.3 Етап 3. Формування замовлення та розрахунок цифрового підпису LiqPay

Для автомобіля ЗАЗ-968М сформовано замовлення «ORD-ЗАЗ-968М-20261030-085514-498»:

- **Сума:** 15.00 UAH.
- **Параметри конкатенації:**

```
sandbox_priv_key_88492040215UAHsandbox_i715474304020RD-ЗАЗ-968M...
```

- **Згенерований підпис форми:** «FsSiswYfooMI9fskCSzvcq7fnDs=».

5.5.4 Етап 4. Симуляція атаки підробки суми (Tampering Attack)

Зловмисник спробував підмінити суму транзакції у Webhook з 15.00 UAH на 1.00 UAH, не знаючи «private_key» магазину:

- **Результат верифікації:** «INVALID (False)».
- **Реакція системи безпеки:** транзакцію відхилено зі статусом «400 Bad Request».
- **Зміна залишку товару:** залишок ЗАЗ-968М залишився рівним **3 шт.** (списання заблоковано).

5.5.5 Етап 5. Успішна оплата через тестовий режим (Sandbox Success Card)

Надіслано тестову транзакцію з валідними реквізитами картки «4242 4242 4242 4242»:

- **Результат верифікації:** «VALID (True)».
- **Статус замовлення:** «Paid».
- **Зміна залишку ЗАЗ-968М: 3 шт. → 2 шт.** (успішно списано 1 одиницю).

5.5.6 Етап 6. Перевірка ідемпотентності (захист від дублювання платежу)

Шлюз повторно надіслав ідентичний Callback для вже оплаченого замовлення:

- **Результат обробки:** система визначила статус «Paid», списання товару вдруге не відбулося.

– **Залишок ЗАЗ-968М:** залишився рівним **2 шт.**

5.5.7 Етап 7. Неуспішна оплата (Sandbox Failure Card)

Сформовано друге замовлення на ЗАЗ-968М та здійснено спробу оплати відхиленою карткою «4000 0000 0000 0002»:

– **Статус замовлення:** «Failed».
– **Зміна залишку товару: False**.
– **Залишок ЗАЗ-968М:** залишився рівним **2 шт.** (після неуспішної оплати товар не списується).

Сценарій тесту	**Картка**	**Підпис**	**Склад ЗАЗ**	**Результат аудиту**
Початковий стан	–	–	3 шт.	Склад готовий до торгівлі
Спроба зламу (Tampering)	Підроблена	Невалідний	3 шт.	Спроба підробки 15 -> 1 грн відхилена
Успішна оплата	4242 (Success)	Валідний	2 шт.	Списано 1 авто (3 -> 2 шт.)
Повторний Webhook	4242 (Success)	Валідний	2 шт.	Ідемпотентність: подвійне списання відхилено
Відхилена оплата	4000 (Declined)	Валідний	2 шт.	Відмова банку: залишок не змінився (2 шт.)

5.6 Відповіді на контрольні запитання

1. Поясніть поняття «банк-еквайєр» і «банк-емітент».

– **Банк-еквайєр (Acquirer):** комерційний банк, який обслуговує торговельне підприємство (інтернет-магазин). Він надає технічну та фінансову можливість приймати безготівкові платежі, відкриває торговцю мерчант-рахунок, маршрутизує платіжні запити в платіжні системи (Visa, Mastercard) та зараховує кошти на розрахунковий рахунок магазину.
– **Банк-емітент (Issuer):** комерційний банк, який випустив платіжну картку покупцеві на основі укладеного договору банківського рахунку. Банк-емітент зберігає кошти покупця, здійснює його ідентифікацію та автентифікацію (зокрема, через протокол 3-D Secure OTP), приймає рішення про схвалення або відхилення транзакції та несе відповідальність за авторизоване списання коштів.

2. Поясніть покроково схему купівлі товару в інтернет-магазині.

а) **Вибір товару:** покупець додає товар до кошика на сайті торговця та переходить до оформлення замовлення.

б) **Ініціалізація замовлення на сервері торговця:** бекенд магазину створює запис замовлення з унікальним «order_id», фіксує суму, генерує цифровий підпис на базі секретного «private_key» та формує платіжну форму.

в) **Перенаправлення на платіжний шлюз:** браузер користувача направляє платіжні параметри до шлюзу (LiqPay).

г) **Введення реквізитів картки:** покупець вводить номер картки (PAN), термін дії та CVV2/CVC2 код у захищеному інтерфейсі шлюзу (відповідно до вимог стандарту безпеки PCI DSS).

д) **Авторизація та 3-D Secure:** платіжний шлюз звертається до платіжної системи, яка адресує запит банку-емітенту. Покупець підтверджує платіж одноразовим кодом у додатку або SMS.

е) **Списання та сповіщення торговця (Server-to-Server Callback):** банк-емітент резервує кошти, шлюз генерує новий цифровий підпис та асинхронно викликає API торговця («server_url»).

ж) **Верифікація підпису та завершення:** сервер магазину перевіряє підпис, переводить замовлення у статус «Paid», зменшує складські залишки та перенаправляє покупця на сторінку успішної оплати («result_url»).

3. Як уникнути багаторазової покупки того самого товару?

Для уникнення подвійного або повторного списання товару застосовують комплекс технічних заходів:

– **Принцип ідемпотентності (Idempotency):** кожне замовлення отримує глобально унікальний ідентифікатор «order_id» (наприклад, поєднання префікса, UUID або мітки часу). Обробник callback-повідомлень перед зміною залишків перевіряє поточний стан замовлення. Якщо транзакція з цим «order_id» вже перебуває у стані «Paid», повторна зміна складу суворо блокується, а сервер повертає статус успішного прийому без повторних дій.

- **Тимчасове резервування (Hold/Reserve):** під час створення форми замовлення товар може тимчасово резервуватися на визначений час (наприклад, 15 хвилин). Якщо оплата не надійшла вчасно, резерв автоматично знімається.
- **Атомарні транзакції баз даних:** зміна лічильника залишку на складі повинна виконуватися на рівні атомарної операції СКБД (із рівнем ізоляції «Serializable» або умовою «WHERE stock > 0») для виключення стану гонитви (Race Condition) при одночасній оплаті кількома покупцями.

4. Як можна дізнатися, що покупець купив товар або відмовився від покупки?

Існує три основних механізми отримання статусу:

а) **Асинхронний серверний Callback / Webhook («server_url»):** найнадійніший канал сповіщення за схемою «сервер-сервер». Платіжний шлюз гарантовано доставляє статус транзакції («success», «failure», «wait_secure») безпосередньо бекенду магазину, незалежно від того, чи закрив покупець вкладку браузера після оплати.

б) **Клієнтська переадресація («result_url»):** після оплати користувач перенаправляється назад до сайту магазину. Проте цей канал вважається ненадійним для фінансового обліку, оскільки клієнт може втратити інтернет-з’єднання або закрити вікно.

в) **Активне опитування API платіжного шлюзу (Polling / Status Check API):** за необхідності сервер магазину може самостійно надіслати запит до API шлюзу (наприклад, метод «status» у LiqPay) за «order_id» для звірки поточного стану платежу.

5. Які поля включає в себе форма платіжного серверу?

Відповідно до документації LiqPay платіжна форма містить:

- **Обов’язкові поля:**
  - «public_key» – публічний ідентифікатор мерчанта в системі.
  - «amount» – сума до списання (число з рухомою комою).
  - «currency» – трилітерний код валюти за стандартом ISO 4217 (UAH, USD, EUR).
  - «description» – текстовий опис придбаного товару чи послуги.

- **Рекомендовані поля безпеки та маршрутизації:**
  - «order_id» – унікальний ідентифікатор замовлення в інтернет-магазині.
  - «type» – тип фінансової операції («buy» – купівля, «donate» – благодійна пожертва).
  - «result_url» – URL для повернення покупця після завершення оплати.
  - «server_url» – API-ендпоінт магазину для прийому підписаних повідомлень про статус.
  - «signature» – криптографічний підпис параметрів (Base64 SHA-1).
  - «language» – мова інтерфейсу платіжного кабінету («uk», «en»).

6. Яка різниця між тестовим та реальним режимом роботи платіжної системи?

- **Тестовий режим (Sandbox / Test Mode):** призначений для повної наладки програмної взаємодії, відпрацювання сценаріїв успіху, відмови, тайм-аутів та перевірки підписів. У цьому режимі використовуються тестові API-ключі («sandbox_...») та спеціальні тестові номери карток (наприклад, «4242...» для успіху або «4000...» для відхилення). Грошові кошти з реальних карток не списуються, а транзакції не передаються до банківського клірингу.
- **Реальний режим (Production):** вимагає укладання юридичного договору з банком-еквайєром, проходження верифікації торговця (KYC/KYB), надання зареєстрованого доменного імені та комерційних API-ключів. Транзакції супроводжуються реальним блокуванням та перерахуванням коштів між банківськими рахунками за участю банків-емітентів і міжнародних платіжних систем із нарахуванням комісії за еквайринг.

5.7 Висновки

У ході виконання лабораторної роботи №5 було успішно розроблено та досліджено архітектуру платіжного модуля веб-сайту:

а) За допомогою фреймворку ASP.NET Core Minimal API (.NET 10) реалізовано повноцінний сервіс продажу автомобілів із суворим контролем складських залишків.

б) Виконано індивідуальну вимогу викладача щодо контролю залишків: для автомобіля ЗАЗ-968М зафіксовано початковий залишок 3 шт., який після успішної транзакції зменшується до 2 шт., а при відхиленій оплаті – залишається 3 шт.

в) Реалізовано та протестовано алгоритм криптографічного цифрового підпису транзакцій LiqPay (Base64 SHA-1), доведено його надійність у захисті від атак підробки та підміни параметрів платежу (Tampering).

г) Забезпечено захист від повторних списань на основі принципу ідемпотентності замовлень за унікальним «order_id».

д) Успішно імплементовано протокол безпечної авторизації клієнтів за стандартом OAuth 2.0 (RFC 6749) з отриманням токенів Bearer, що задовольнило критерій оцінювання на рівні «відмінно».

5.8 Додаток А. Посилання на вихідний код у репозиторії GitHub

Вихідний код лабораторної роботи №5 розміщено у публічному репозиторії GitHub:

- Головний файл веб-сервера Minimal API: lab5/src/server/Program.cs
- Сервіс криптографічних підписів LiqPay: lab5/src/server/Services/LiqPayService.cs
- Сервіс складського обліку та ідемпотентності: lab5/src/server/Services/InventoryService.cs
- Провайдер автентифікації OAuth 2.0 (RFC 6749): lab5/src/server/Services/OAuthService.cs
- Клієнтська частина та платіжний термінал: lab5/src/server/wwwroot/index.html
- Інтерактивна логіка та тестові картки: lab5/src/server/wwwroot/app.js

– Стилі веб-додатка: lab5/src/server/wwwroot/style.css